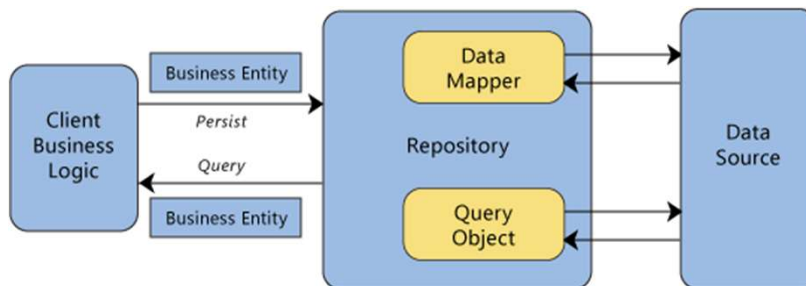


Repository Pattern

Schnittstelle zwischen Domäne und Datenzugriff

Repository

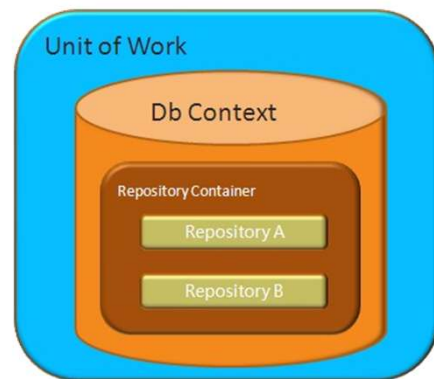
- Wichtiger Bestandteil des DDD
- Abstrahiert Datenzugriff hinter dem sog. Repository
- ICustomerRepository (mit Zugriffsmethoden)
- Besser: IRepository<T>



Das Repository-Muster basiert auf der Idee, dass Datenzugriffscode vom Geschäftslogikcode getrennt werden sollte. Das Repository-Muster **bietet eine Möglichkeit, Datenzugriffscode an einem zentralen Ort zu verwalten, wodurch Code-Duplikationen reduziert und die Wartbarkeit des Codes verbessert werden.**

Unit of Work

- Abstrahiert Persistenz-Operationen an Repositories (transaktional) in einer eigenen Klasse
- Normalerweise ein Interface,
 - IDisposable
 - Enthält intern DbContext (oder zugeführt)
 - Methode Save, Commit, RollBack



CQRS

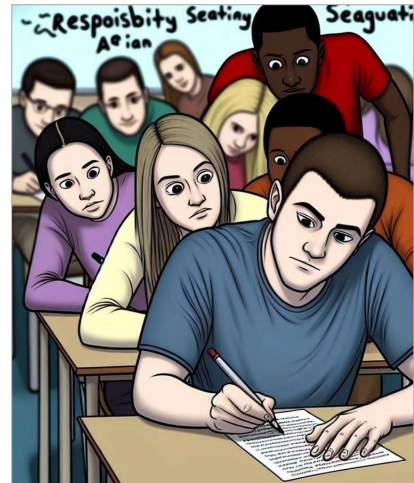
Command-Query-Responsibility-Segregation

CQRS & Event-Sourcing

- Zwei unabhängig Konzepte
- Aber ergänzen sich sehr gut

CQRS: Command-Query-Responsibility-Segregation

- Trennung von Commands und Queries
- Nur zwei Zugriffsformen: Lesen oder Schreiben
- Anwendung teilen in
 - Lesende Vorgänge
 - Schreibende Vorgänge
 - Implementierung dabei völlig offen!



Wie die Trennung implementiert wird, ist völlig offen!

<https://www.youtube.com/watch?v=hP-2ojGfd-Q>

Warum sollten wir trennen?

- Datenbank ist die Wurzel des Elends
- Hat großen Einfluss auf Geschäftslogik und letztlich API

Wie sieht ein gutes Datenmodell aus?

- Normalform **5** -- Bestes Model zum Schreiben
- Normalform **1** -- Bestes Model zum Lesen
- Normalform **3** -- Als Kompromiss

Lösung: Zwei Datenmodelle

- Optimiert auf das Schreiben
- Optimiert auf das Lesen

Synchronisation?

Warum gibt es die Empfehlung von CQRS überhaupt?

- Dreh- und Angelpunkt ist die Datenbank
- Hat große Auswirkungen darauf, wie die Geschäftslogik letztlich für API implementiert wird
- Daher die Frage: Wie sieht ein gutes Datenmodell aus?
- > 5 Normalformen

5. Normalform ist das Beste Schreibmodell

- Aber benötigt viele Joins, daher möglicherweise sehr langsam zu lesen
- Sie ist eine Katastrophe beim Lesen

1. Normalform das Beste Lesemodell

- Beste Performance beim Lesen
- Aber redundante Datenhaltung
- Konsistenz und Integrität?

3. Normalform als Kompromiss

Lösung:

Zwei Datenmodelle

Herausforderung:

Änderungen zwischen beiden Modellen synchron halten (später mehr)

Verleih-Service mit Event-Sourcing

- Event-Sourcing ist **Prozessgetrieben** statt **Datengetrieben**
- Prozesse des Verleihs lassen sich hervorragend als **Commands** abbilden
- Datenbanktabelle mit **Event(-Logs)** bildet Schreibmodell ab
- Dedizierte Lesemodelle für **Queries** um Fragen zu beantworten
- **Synchronisation**: Listen werden parallel beim Schreiben mit aufgebaut



Event-Sourcing ist Prozessgetrieben statt Datengetrieben
(Schwerpunkt auf Verben statt Substantiven)

Welche Prozesse gibt es beim Verleih-Service?

- Fahrrad ausleihen
- Fahrrad zurück geben
- Mietdauer verlängern
- Beschädigung des Fahrrads

-> Lassen sich hervorragend in Commands abbilden

Fahrrad im System suchen > Verfügbar? > Button "Fahrrad leihen"

-> Genau das ist der Command: Fahrrad leihen

- System führt Geschäftslogik aus und befolgt Business Rules
 - Bsp. darf ich ein Premium-E-Bike überhaupt ausleihen?
 - Zustand des Systems ändert sich und Schreibvorgang wird ausgelöst
 - Event-Log bietet sich für das Erfassen der Änderungen sehr gut an
 - >> **Event-Sourcing**: Das ist unser Schreibmodell
- Service möchte Stats wissen
 - Was ist gerade ausgeliehen?
 - Welches Modell war letztes Jahr am beliebtesten?

- Gibt es Modelle, die nicht verliehen wurden?
- Wieviel % wurden beschädigt?
- Welche Transportmittel wurden geklaut?
- >> **Fragen lassen sich auf Basis** von Events beantworten

- Durch Liste aller Events iterieren beantwortet die Frage
- Problem: Aufwand und Performance
- Lösung: Lesemodelle (Listen/Tabellen) werden parallel mit aufgebaut

- Wenn ein Transportmittel ausgeliehen wird, schreiben wir das auch in eine Verleih-Tabelle
- Wenn es zurück gegeben wird, entfernen wir es wieder
- Beim Verlängern passiert gar nichts
- Lesemodelle lassen sich durch Replays dynamisch wiederaufbauen, ergänzen, entfernen...

- Schreibmodell als **Single-Source-Of-Truth**

Verschiedene Lesemodelle für Verleih-Service

- Liste der ausgeliehenen Transportmittel
 - Relationale DB, NoSQL, usw.
- Lesemodelle sind unabhängig von der Technologie
 - Bsp. Volltextsuche mit Elastic Search
- Lesemodelle müssen nicht persistiert werden
 - Im RAM bedeutet es einen extra Performance-Boost
 - Bsp. [Memgraph](#), [DuckDB](#)
- Lesemodelle je nach Permissions (mehr oder weniger Daten)
- Zuggeschnittene Lesemodelle für Externe (Data Mesh)



- Nicht jeder Lesevorgang hat dieselben techn. Anforderungen
- Bsp. Liste der ausgeliehenen Transportmittel: Relationale DB oder NOSQL
- Wenn ich Volltextsuche haben möchte, dann bräuchte ich eine Elastic-Search
- Lesemodelle sind technologieunabhängig (Die eine Tabelle in der DB, die andere woanders)
- Müssen nicht mal persistiert werden und könnten im RAM gehalten werden wie Memgraph, DuckDB
- Gutes Fundament, um Data-Mesh für Externe Interessenten bereit zu stellen

Herausforderungen bei Lesemodellen

Kann die Lese-API der Flaschenhals werden?

- Nein, es lässt sich auch das gleiche Lesemodell auf verschiedene Instanzen verteilen (zielgerechtes Skalieren)
- CQRS ermöglicht es punktgenau und bedarfsgerecht zu skalieren

Wie kritisch ist die benötigte Zeit für Synchronisation?

- Kommt auf die Umstände an
 - Wahrscheinlichkeit der Latenz, Resultierende Probleme, Risikobewertung
- Möglicher Workaround: Kapselung als Transaktion

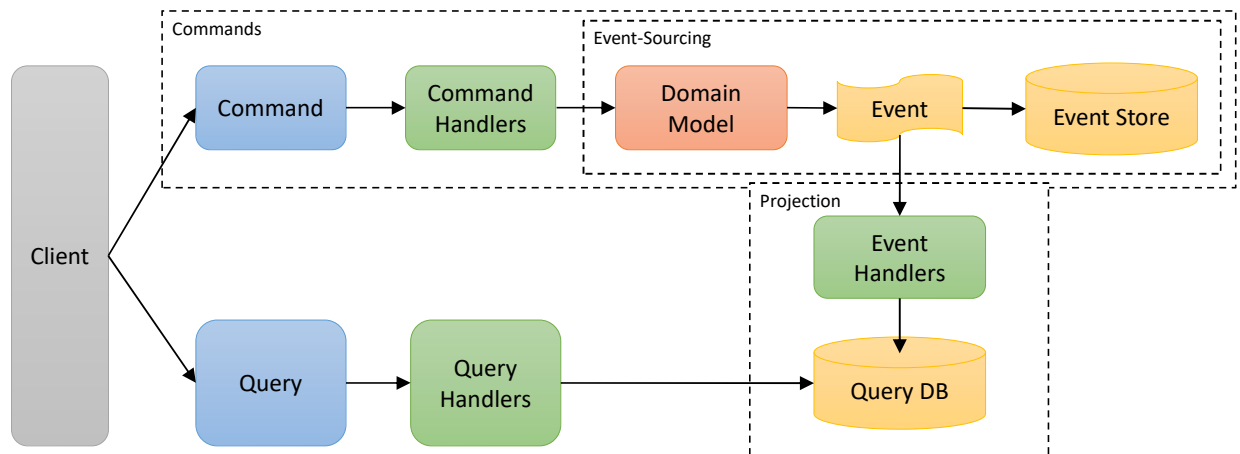
Herausforderungen bei Synchronisation:

- Benötigte Zeit für Synchronisation

Strong consistency vs. eventual consistency

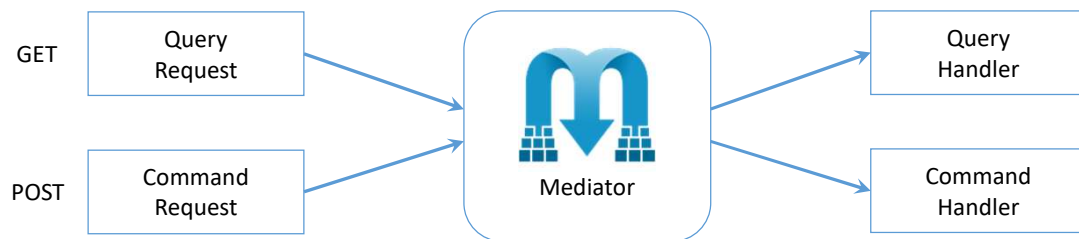
- Wie schlimm ist die Verzögerung in der Praxis?
- > Kommt darauf an: Es ist eine fachliche Frage.
 - Wie hoch ist die Wahrscheinlichkeit der Latenz?
 - Was für ein Problem kann resultieren?
 - Wie hoch ist das Risiko beim Auftreten?
- Mögliche Lösung: Kapselung als Transaktion
 - Kann andere Nachteile haben
 - Transaktionen in verteilten Systemen ist kein Vergnügen
- Es muss herausgefunden werden, ob **strong** oder **eventual** **passender** ist
 - Bsp. Geldabheben am ATM der keine Netzwerkverbindung hat
 - Soll er weiterhin Geld heraus geben?
 - Ja, weil nur geringes Risiko / verursachter Schaden
 - Die meisten Menschen heben nur geringe Beträge ab
 - und wissen ob Konto gedeckt ist

CQRS & Event-Sourcing



CQRS mit MediatR

- IRequest / IReponse
- INotification



Service API

RESTful APIs über das Web

Was ist eine API?

- Ein HTTP Service
- Logik oder Daten sind über das HTTP Protokoll zugänglich
- Wird für die Anwendungsentwicklung benötigt
- Über das Internet zugänglich



REST?

Representational State Transfer
in der Softwareentwicklung
ähnlich wie HTTP oder SOAP
in ressourcenorientierter Architekturstil
Nutzung universeller Interfaces des HTTP (GET, POST, PUT, DELETE)
Nutzung von Ressourcen mittels URI
zustandslos

response: {"id":1,"content":"Hello, World!"}



REST ist Architekturstil, zur Erstellung von verteilten Anwendungen. Dieser beinhaltet die Erstellung einer Resource Orientierten Architektur (ROA), indem die Ressourcen universelle Interfaces mittels der standard HTTP Verben (GET, POST, PUT, DELETE) implementiert. Die Ressourcen können mittels Uniform Resource Identifier (URI) lokalisiert und definiert werden.

Jeder Service, der den REST Architekturstil verwendet, wird als RESTful Service bezeichnet.

REST Zugriff auf Ressourcen (mittels HTTP)

- GET holt die Ressource vom Server ab: *GET* <http://ppedv.de/brezn/123>
- POST erstellt eine Ressource: *POST* <http://ppedv.de/brezn/123>
- PUT verändert (=update) ein Objekt in einer Ressource:
PUT <http://ppedv.de/brezn/123?date=20180209>
- *DELETE* löscht eine Ressource vom Server
- Das Datenformat im GET darf sein: XML, JSON, RSS

Die Bedeutung von Status Codes

LEVEL 200 SUCCESS

200 – OK
201 – Created
204 – No Content

LEVEL 400 CLIENT ERROR

400 – Bad Request
401 – Unauthorized
403 – Forbidden
404 – Not Found
409 – Conflict

LEVEL 500 SERVER ERROR

500 – Internal
Server Error

<https://de.wikipedia.org/wiki/HTTP-Statuscode>